SHAP-guided Robustness Evaluation of Deep Learning-based Smart-home NIDSs

Matteo D'Avino, Luigi Fienga, Andrea Roscigno

Università degli Studi di Napoli Federico II

{matt.davino, lui.fienga, and.roscigno}@studenti.unina.it

***Abstract*—Deep learning-based Network Intrusion Detection Systems (NIDSs) achieve high detection performance in Smart-home IoT environments, but their reliance on complex black-box models raises both interpretability and security concerns. While SHAP explanations can clarify which traffic features drive a model prediction, the same information may also reveal exploitable weaknesses. This work proposes a SHAP-guided robustness evaluation procedure for DL-based Smart-home NIDSs, using explanation-guided feature perturbations as a diagnostic probe rather than as a new adversarial attack. The evaluation is conducted on the ToN-IoT dataset and compares three architectures with different inductive biases: a convolutional model, a Transformer-based model, and a Mamba-based model. We consider both untargeted evasion and the more operationally relevant benign-targeted evasion, under loose and strict feature-level constraint regimes. Results show that SHAP-guided perturbations are more effective than random admissible perturbations, increasing Attack Success Rate (ASR) while reducing the average magnitude of successful changes. However, benign-targeted evasion is substantially harder than generic misclassification and exhibits strong architecture- and class-dependent behavior. NetMamba is the most exposed to SHAP-guided perturbations, whereas BiflowTransformer appears to be more robust in the standard benign-targeted setting. Feature-level analysis further shows that successful evasions are concentrated on a limited set of feature, mainly involving Packet Lenght (PL) and Time To Live (TTL). These findings highlight the dual-use nature of explainability in NIDSs and suggest that SHAP can support not only model interpretation, but also systematic robustness assessment.**

***Index Terms*—Network Intrusion Detection Systems, Smart-home IoT, Explainable Artificial Intelligence, SHAP, Adversarial Robustness, Evasion Attacks, Deep Learning.**

I. Introduction

The rapid growth of Smart-home IoT deployments has considerably expanded the residential attack surface, exposing a heterogeneous ecosystem of resource-constrained and infrequently updated devices to large-scale cyber threats. This risk is not merely theoretical: design flaws in commercial smart-home platforms have enabled privacy-violating overprivilege in third-party applications [1]; large numbers of compromised consumer IoT devices have been recruited into botnets that launched record-scale distributed denial-of-service attacks [2]; and smart-home security investigations have reported thousands of scanning or hacking attempts against consumer devices within short observation windows [3]. Detecting malicious network activity before it escalates into these attacks is therefore a fundamental requirement for securing modern Smart-home environments.

Deep Learning has become a widely explored paradigm for Network Intrusion Detection systems (NIDSs), owing to its ability to automatically learn discriminative representations directly from network traffic and achieve high detection performance [4], [5]. This performance, however, comes at the cost of interpretability: DL-based NIDSs operate largely as black boxes, providing little insight into which characteristics of a network flow drive a particular prediction. To mitigate this limitation, eXplainable Artificial Intelligence (XAI) techniques, and SHapley Additive exPlanations (SHAP) [6], in particular, are increasingly adopted to quantify the contribution of each input feature to the model's decision.

The same transparency that improves model interpretability, however, also introduces a potential security risk. By revealing which features most strongly influence a prediction, SHAP explanations may inadvertently disclose the information an adversary needs to manipulate network traffic and evade detection. While this dual-use nature of XAI has recently attracted attention, an important question remains largely unexplored: can explanation-guided feature manipulations be systematically used to assess the robustness of DL-based NIDSs, and does the resulting vulnerability depend on the underlying learning architecture?

As discussed in Section II, existing studies provide initial evidence that SHAP explanations can facilitate adversarial evasion. However, current evaluations are typically limited to a single learning architecture and predominantly consider untargeted attacks, where any change in the predicted class is regarded as a successful evasion. Although informative, this evaluation does not fully capture the threat faced by a practical NIDS, where an attacker ultimately seeks to have malicious traffic accepted as benign rather than simply misclassified into another attack category.

To address these limitations, this paper proposes a SHAP-guided robustness evaluation procedure for DL-based Smart-home NIDSs. Rather than introducing a new adversarial attack, we use explanation-guided feature perturbations as a principled probe to quantify how vulnerable different architectures are to manipulations of the features they themselves identify as most influential. Our evaluation considers two complementary threat models, both assessed under loose and strict feature-level constraint regimes. The first investigates untargeted eva-

sion by measuring whether perturbing the most influential features of each predicted class is sufficient to induce any misclassification. The second introduces a benign-targeted evaluation, in which perturbations are guided by the features most characteristic of the benign class to assess whether malicious traffic can be systematically manipulated to mimic legitimate behavior.

We instantiate the proposed procedure on the ToN-IoT dataset [7] using three deep learning architectures with fundamentally different inductive biases: a convolutional neural network (LopezCNN) [8], a self-attention-based Transformer (BiflowTransformer), and a selective state-space model (NetMamba) [9]. This comparative evaluation enables us to investigate whether explanation-guided exploitability represents a general property of DL-based NIDSs or whether it is significantly influenced by the underlying architectural design.

The remainder of this paper is organized as follows. Section II reviews related work on Smart-home network intrusion detection, explainable artificial intelligence for cybersecurity, and adversarial attacks against deep learning-based NIDSs. Section III presents the proposed methodology and robustness evaluation of procedure. Section IV reports the experimental setup, while Section V presents the result of this work. Section VI concludes the paper and discusses future research directions.

II. Related work

A. Smart-home and IoT Network Intrusion Detection

The increasing deployment of IoT and Smart-home devices has motivated extensive research on network intrusion detection systems designed for heterogeneous and resource-constrained environments [7], [10]. Deep learning has been widely adopted for traffic classification and intrusion detection because it can learn discriminative representations directly from network traffic features [8].

B. Explainable AI for NIDs

Deep learning-based NIDSs can achieve high detection performance, but their decision processes often remain difficult to interpret because ML/DL models are opaque and affected by explainability issues [11] [12] [13]. To address this limitation, Explainable Artificial Intelligence (XAI) techniques are used to clarify model decisions by attributing importance to input features; in particular, SHAP assigns each feature an importance value for a given prediction [6]. In IoT intrusion detection, Sharma et al. applied SHAP to explain DL-based IDS models [11], while Siganos et al. used SHAP to explain the decision-making of an AI-powered IoT IDS [12]. More broadly, Cerasuolo et al. report that SHAP is widely used in NIDS studies to evaluate feature importance from both global and local perspectives [13].

Beyond interpretability, XAI has also been used for robustness-oriented analysis. Barnard et al. [14] proposed a two-stage framework in which SHAP explanations are used to train an autoencoder capable of distinguishing between previously seen and unseen attacks, enabling the detection of new attacks during testing [14].

However, feature-attribution methods can also be dual-use: by revealing the input features that most strongly influence model decisions, XAI may expose information that adversaries can exploit to craft targeted perturbations and evade DL-based NIDSs [15] [16].

C. Adversarial Attacks Against DL-based NIDS

Deep learning-based NIDSs have been shown to be vulnerable to adversarial evasion attacks, where carefully crafted perturbations can significantly degrade detection performance [17] [18]. Prior studies have demonstrated that such attacks are effective across different model families, including sequential architectures, highlighting the susceptibility of DL-based intrusion detectors to input manipulation [17] [18]. A key aspect in this line of research concerns the realism of adversarial perturbations. Since network traffic features are not all independently controllable, several works have introduced constraint-aware settings that restrict modifications to semantically valid or attacker-feasible features [18] [19] [20]. These constraints aim to reduce the gap between purely theoretical feature-space attacks and practical adversarial scenarios, where perturbations should preserve flow consistency and operational feasibility [19] [20]. Overall, these studies establish that DL-based NIDSs are vulnerable to evasion attacks, including under constrained perturbation settings, but they do not investigate whether the choice of features to perturb can be systematically guided by model explanations.

D. XAI-guided Evasion and Explanation Leakage

More recent work has investigated the intersection between explainability and adversarial machine learning in intrusion detection systems. Building on this line of research, several studies have shown that feature attribution methods such as SHAP and LIME can be exploited to guide adversarial perturbations by identifying the most influential input features for model decisions [15] [16] [21]. Okada et al. proposed XAI-driven black-box attacks using KernelSHAP to approximate influential features without requiring internal model access [15], while Mangiacapre et al. demonstrated that SHAP explanations can significantly reduce the search space for effective evasion strategies [16]. Similarly, Chung and Han [22] introduced a DeepSHAP-based attack strategy targeting the most influential features identified by explanation methods.

In addition to attack design, explanation-guided feature selection has been used to improve the efficiency of adversarial example generation and to analyze vulnerabilities in learning-based NIDSs [21] [23]. These results consistently indicate that explanations can act as an enabler for adversarial evasion by revealing model-sensitive input dimensions [15] [16] [21] [22] [23].

E. Positioning of This Work

Existing XAI-guided attack studies, summarized in Table I, primarily leverage explanations to improve adversarial evasion

TABLE I
POSITIONING OF THIS WORK WITH RESPECT TO XAI-GUIDED ADVERSARIAL ATTACKS AGAINST NIDS. THE LAST ROW SUMMARIZES THE PRESENT WORK.

Paper	Year	Dataset	Task setting	Models evaluated	XAI-Method	XAI-guided attack objective	Multi-class benign-targeted eval.	Comparative DL robustness
Okada et al. [15]	2025	CIC-IDS2017, TON_IoT	B	MLP-based NIDS	SHAP	$\rightarrow B$	×	×
Li et al. [21]	2025	CIC-IDS2017, Kitsune	B	RF, SVM, MLP, KitNET	SHAP & LIME	$\rightarrow B$	×	△
Mangiacapre et al. [16]	2026	InSDN	B	2D-CNN	SHAP	$\rightarrow B$	×	×
Chung and Han [22]	2026	X-IIoTID	M	MLP	SHAP	$\star$	×	×
This work	*2026*	*ToN-IoT*	M	*Lopez17CNN, Bi-flowTransformer, NetMamba*	*SHAP*	$\star \rightarrow B$	✓	✓

Task setting: B = binary benign/malicious; M = multi-class.
XAI-guided attack objective: $\rightarrow B$ = binary malicious-to-benign evasion; $\star$ = multi-class untargeted evasion; $\star \rightarrow B$ = multi-class untargeted plus malicious-to-benign targeted evasion.
Symbols: ✓ (yes); × (no); △ (partial).

effectiveness, for example by identifying highly influential traffic features to perturb [15], [16], [21], [22]. However, prior evaluations often consider either binary benign/malicious detection settings, where malicious-to-benign evasion is naturally induced by the binary label space [15], [16], [21], or untargeted multi-class attacks, where any misclassification is considered successful and targeted variants are not systematically assessed [22]. This leaves open the practically relevant multi-class question of whether explanation-guided perturbations can consistently force different malicious traffic classes to be classified specifically as benign. Addressing this gap, our work evaluates both untargeted and explicit benign-targeted SHAP-guided perturbations in a six-class Smart-home NIDS setting, using explanations as a principled probe to compare the robustness of deep learning architectures with different inductive biases.

III. METHODOLOGY

A. Procedure Overview

This work proposes a SHAP-guided robustness evaluation procedure for deep learning-based Smart-home NIDSs. Given a trained NIDS model (f_m), where m denotes the considered architecture, the procedure first identifies the network flows that are correctly classified by all models under comparison. These common correctly classified samples are then explained using SHAP, and the resulting feature attributions are aggregated to obtain class-level importance profiles. Finally, these importance profiles are used to guide controlled feature perturbations and evaluate whether the model prediction can be changed, either toward any incorrect class or specifically toward the benign class. The goal of the proposed procedure is not to introduce a new adversarial attack algorithm. Instead, SHAP-guided perturbations are used as a principled probe to assess how vulnerable different architectures are to manipulations of the features that they themselves identify as most relevant. This allows us to compare explanation-guided exploitability across models with different inductive biases.

B. SHAP-based Explanation and Feature Ranking

We adopt SHAP GradientExplainer [24] to attribute predictions to input features, as it estimates Shapley values via Expected Gradients and applies uniformly to any differentiable model. This property is central to a cross-architecture study such as ours: CNNs, Transformers, and Mamba-based models expose very different internal computations (e.g., attention heads and LayerNorm blocks in the Transformer case), and architecture-specific attribution methods such as Layer-wise Relevance Propagation would require dedicated propagation rules for each of them [25]. Using a single explanation procedure across all three architectures removes this confound and lets us attribute observed differences in exploitability to the model rather than to the explanation method.

Raw SHAP attributions, however, are not directly usable as an attack signal: their scale varies from sample to sample, and any single sample's ranking is noisy. We therefore aggregate them into a class-level importance profile. For every correctly classified sample, we normalize its feature SHAP attribution map by the sum of its absolute values, so that attributions become comparable in relative rather than absolute magnitude, following the aggregation strategy adopted in [16]. We then take the median, rather than the mean, of these normalized maps across all samples of the same class, so that the resulting profile is not skewed by individual samples with atypically large SHAP values. The result is, for each model m and class c, a matrix $\Phi^{m,c}$ whose entries $\Phi_j^{m,c}$ quantify how consistently the feature j contribute to the model's decision for class c across the population of samples to obtain a stable, population-level attribution rather than a single explanation, and it is this matrix that drives the perturbations described next.

$\Phi^{m,c}$ is used differently depending on the threat model. For untargeted evasion, we rank feature by the absolute importance of the currently predicted class,

$$R^{m,c}_{\text{untargeted}} = \text{TopK}_j \left(\left| \Phi^{m,c}_j \right| \right), \tag{1}$$

capturing the features the model leans on most, in either direction, to sustain its own prediction. For benign-targeted evasion, we instead rank by the positive contribution to the benign class,

$$R^{m,\text{benign}}_{\text{targeted}} = \text{TopK}_j \left(\Phi^{m,\text{benign}}_j \;\middle|\; \Phi^{m,\text{benign}}_j > 0 \right), \tag{2}$$

i.e., the directions the model itself associates with legitimate traffic. In both cases the ranking is restricted a priori to the subset of feature an attacker could plausibly modify (Section III-C), so that SHAP only decides which admissible feature to prioritize.

C. Perturbation Model and Constraint Regimes

All perturbations are generated in the physical feature domain and then mapped back to the normalized input representation expected by the models. In all experiments, at most one feature is modified per trial, always starting from the original unmodified sample; therefore, perturbations are not accumulated across attempts. For each selected feature, candidate values are generated in the physical domain: perturbations explore stepwise variations around the original value and retain in the first successful value.

We define two constraint regimes. Loose constraints are applied to all experiments and restrict perturbations to admissible feature-level modifications in the physical traffic domain. Under this regime, non-modifiable or categorical fields are not altered, padded positions are ignored, and candidate values are kept within valid physical ranges.

Strict constraints further reduce the perturbation space to approximate a more realistic attacker. In this regime, only a subset of otherwise admissible modifications is allowed, following feature-level feasibility assumptions used in prior adversarial studies on sequential traffic models [18]. This setting evaluates whether SHAP-guided evasion remains effective when the attacker is limited to more plausible traffic manipulations.

D. Untargeted and Benign-Targeted Evasion Objectives

The two threat models introduced above translate directly into two evasion objectives. Untargeted evasion asks whether a malicious sample correctly classified as its true attack class c can be pushed to any other class,

$$f_m(x') \neq c, \tag{3}$$

and measures how easily a model's decision boundary can be destabilized. Benign-targeted evasion is stricter: starting again from a malicious sample correctly classified as c, success requires the perturbed sample to be classified specifically as benign,

$$f_m(x') = \text{benign}. \tag{4}$$

This second objective is the one that matters operationally, since an attacker's actual goal is not to be reclassified into a different attack category, but to have malicious traffic accepted as legitimate.

E. Evaluation Scenarios

Crossing the two objectives with the two constraint regimes yields four SHAP-guided scenarios, complemented by a random baseline that isolates the contribution of explanation guidance itself. Under loose constraints, we evaluate: (i) SHAP-guided untargeted evasion, where the top-ranked features of the original attack class ($R^{m,c}_{\text{untargeted}}$) are perturbed one at a time and success is any change away from c; and (ii) SHAP-guided benign-targeted evasion, where perturbations follow the benign-class ranking ($R^{m,\text{benign}}_{\text{targeted}}$) and success requires the sample to be reclassified as benign. The same two SHAP-guided objectives are then repeated as two additional experiments under strict constraints: (iii) SHAP-guided untargeted evasion and (iv) SHAP-guided benign-targeted evasion, to test whether explanation-guided exploitability survives a more realistic manipulation budget.

Taken together, these scenarios define a progressive robustness evaluation: from a random baseline, to SHAP-guided decision-boundary instability, to benign-targeted operational evasion. Repeating the SHAP-guided objectives under strict constraints further tests whether such vulnerabilities persist when perturbations are limited to more realistic attacker-feasible manipulations.

F. Evaluation Metrics

We evaluate model performance before perturbation using macro-accuracy and macro-F1 score. In both cases, the metric is computed as the average of class-wise scores: macro-accuracy averages the accuracy obtained for each class, whereas macro-F1 averages the F1 score obtained for each class. By assigning equal weight to all classes, these metrics account for class imbalance in the test set. Adversarial robustness is evaluated using two complementary metrics. The first is the Attack Success Rate (ASR), defined as the fraction of evaluated samples for which the perturbation satisfies the attack objective. ASR is computed per class and macro-averaged over the malicious classes for aggregate comparisons. The second metric is the Average Minimum Successful Perturbation $(AMSP)$, which measures the average normalized magnitude of the successful perturbations found by the attack procedure. For each successful adversarial example (x'_i), obtained by modifying feature (j), the normalized perturbation is defined as

$$\delta_i = \frac{\left| x'_{i,j} - x_{i,j} \right|}{x^{\max}_j - x^{\min}_j}. \tag{5}$$

$AMSP$ is then computed by averaging this quantity over the set of successful attacks:

$$AMSP_c = \frac{1}{|\mathcal{S}_c|} \sum \delta_i \quad i \in \mathcal{S}_c \tag{6}$$

where $(\mathcal{S}_c)$ denotes the successful adversarial examples for class (c). Lower $AMSP$ values indicate that successful evasion requires smaller feature-level modifications, whereas higher values indicate that larger changes are needed. If no successful adversarial example is found, $AMSP$ is reported as not applicable. To provide a compact summary of attack effectiveness, we additionally report an Evasion Vulnerability Score (EVS), defined as the macro-average of

$$EVS = \frac{1}{|\mathcal{C}|} \sum ASR_c \cdot (1 - AMSP_c) \quad c \in \mathcal{C} \tag{7}$$

across the evaluated classes. This score is not intended to replace ASR and AMSP, but to summarize the joint effect of frequent and low-magnitude successful evasions: higher values indicate that a model is more easily evaded, both because attacks succeed more often and because successful perturbations require smaller normalized changes.

IV. Experimental Setup

This section describes the experimental setup used to instantiate the proposed SHAP-guided robustness evaluation procedure. We first introduce the dataset and preprocessing pipeline, then describe the evaluated DL architectures and training configuration. Finally, we report the SHAP settings, perturbation parameters, and implementation details used for the adversarial robustness evaluation.

A. Dataset and Preprocessing

The experimental evaluation is conducted on the publicly available ToN-IoT dataset [7], which contains heterogeneous IoT/IIoT network traffic collected in a realistic testbed including normal activity and multiple cyber-attacks.

Following the preprocessing and biflow representation strategy adopted in [16], raw ToN-IoT traffic is partitioned into biflows, and each packet is described by six features: Packet Length (PL), Inter-Arrival Time (IAT), packet Direction (DIR), TCP Window Size (WIN), Time-To-Live (TTL), and TCP Flags (FLG). Since WIN and FLG are TCP-specific fields, they are assigned a value of -1 for UDP packets. To support early attack detection, these features are extracted from the first 10 packets of each biflow.

Starting from the resulting ToN-IoT class representation, we aggregate the original labels into six broader class families, as reported in Table II.

TABLE II
Traffic classes considered in the experimental evaluation.

Label	Type	Original ToN-IoT labels
benign	Legitimate traffic	benign
dos	Denial-of-Service attack	dos-http, dos-tcp, dos-tcp-syn
infog-scan-port	Information-gathering / port-scanning attack	infog-scan-port
inject	Injection attack	inject-sql, inject-xss
malwr	Malware traffic	malwr-backd-tcp, malwr-brutef-http, malwr-brutef-ftp, malwr-ransomware
mitm-arpspoof	Man-in-the-middle ARP spoofing attack	mitm-arpspoof

B. Model Architectures

We evaluate three deep learning architectures with different inductive biases. The first model is LopezCNN, a convolutional architecture inspired by prior CNN-based traffic classification models. It consists of two convolutional blocks with 32 and 64 filters, respectively, each followed by pooling and batch normalization, and a fully connected classification head. The model has 786,528 trainable parameters. The second model is BiflowTransformer, a self-attention-based architecture. Features are first mapped to a 128-dimensional latent space through a linear projection, then combined with positional encoding and a learnable classification token before being processed by four Transformer encoder layers with four attention heads. The model has 1,019,390 trainable parameters. The third model is NetMamba, a selective state-space architecture based on Mamba [26]. It projects packet-level inputs to a 128-dimensional sequence representation, processes them through four Mamba blocks, and applies final normalization, mean pooling, and a fully connected classification head. The model has 485,382 trainable parameters.

C. Training Configuration

All models are trained on the same ToN-IoT six-class task using the same packet representation, feature set, class order, and data split. Shorter biflows are padded to obtain fixed-length inputs, while features are min-max normalized before being provided to the models. The resulting input has shape 10×6, corresponding to the first ten packets and the six selected traffic features.

The three architectures process this common input representation differently. LopezCNN operates directly on the feature matrix through convolutional layers. BiflowTransformer and NetMamba first map each level feature vector to a latent representation through a linear input projection before applying, respectively, Transformer encoder layers and Mamba blocks. This ensures that all models receive the same traffic representation while preserving their architecture-specific inductive biases.

All models are trained from scratch and then loaded from the corresponding checkpoints for the robustness evaluation. Training uses the same seed and validation strategy across architectures, with a 10% validation split during data loading. Unless otherwise specified by the saved run arguments, the training framework uses 200 epochs, batch size 64, an initial learning rate of 0.1, a minimum learning rate of 10^{-4}, a learning-rate reduction factor of 3, and patience of 20 epochs. After loading each checkpoint, predictions are computed on

the aligned test set and mapped back to the canonical class order.

D. SHAP Configuration

SHAP values are computed using SHAP 'GradientExplainer', consistently across all three differentiable PyTorch models. Explanations are computed on class probabilities: models that already output probabilities are used directly, while models producing logits are wrapped with a softmax layer before applying SHAP. For each architecture, the background distribution is built by sampling 100 training instances per class, yielding a balanced background set. SHAP explanations are computed on the common correctly classified test samples shared by all three models. For each class, we explain up to 200 samples shared by all models; if fewer than 200 common correctly classified samples are available, all available common samples are used. This ensures that SHAP profiles are always computed on the same physical flows across architectures, under equivalent input conditions. The resulting class-level SHAP matrices are used to rank feature for the adversarial evaluation.

E. Perturbation and Attack Configuration

All attack configurations are evaluated only on malicious samples correctly classified by the considered model. Perturbations are generated in the physical feature domain and then mapped back to the normalized input space. In all cases, feature are modified independently, always starting from the original sample; hence, perturbations are not cumulative.

The perturbation space follows the two constraint regimes defined in Section III. Under loose constraints, only PL, IAT, WIN, and TTL can be modified, while DIR, TCP flags, and padded positions are never altered. Candidate values are clipped to their admissible physical ranges: PL $[1, 1500]$, IAT $[0.001, 60000]$, WIN $[0, 65535]$, and TTL $[0, 255]$. Under strict constraints, perturbations are further limited to packets sent by the hypothetical attacker, identified by $DIR = 0$, and PL and IAT are allowed to increase only [18].

For the SHAP-guided configurations, the top-3 admissible features are selected according to the ranking associated with the considered scenario: the original attack-class ranking for untargeted evasion and the benign-class ranking for benign-targeted evasion. Each selected feature is perturbed one at a time for up to $MAX_STEPS = 1000$, exploring candidate values around the original physical value. The retained perturbation is the minimum one satisfying the scenario-specific success condition: any change from the original class for untargeted evasion, and classification as benign for benign-targeted evasion. The random baseline follows the same perturbation procedure and success criterion as the untargeted setting, but replaces SHAP-based feature selection with random selection. To obtain a more stable estimate, the experiment is repeated for 10 independent rounds, each time selecting three admissible features at random.

TABLE III
CLEAN CLASSIFICATION PERFORMANCE ON THE SIX-CLASS TON-IOT TASK.

Model	Class-avg. Accuracy (%)	Macro-F1
LopezCNN	79.32	0.7466
BiflowTransformer	98.70	0.9871
NetMamba	99.24	0.9871

V. RESULTS

Preliminary comparisons between the SHAP-selected evaluation subset and the random evaluation subset showed largely overlapping robustness trends across the considered models and scenarios. This suggests that the SHAP explanation set captures representative model behavior and that the resulting class-level profiles are not biased toward a peculiar subset of samples. Therefore, to keep the presentation compact, the results reported in the following sections refer to the random evaluation subset.

A. Clean Classification Performance

Table III reports the clean classification performance of the three architectures on the six-class ToN-IoT task. BiflowTransformer and NetMamba achieve almost identical macro-F1 scores, indicating highly balanced performance across traffic classes. NetMamba obtains the highest class-averaged accuracy, while BiflowTransformer remains very close. LopezCNN shows a less balanced behavior, although it performs well on classes such as dos, inject, and malwr, mainly due to poor performance on the mitm-arpspoof and infog-scan-port classes. These clean results provide the reference point for the robustness evaluation: subsequent adversarial experiments are applied only to malicious samples correctly classified by the considered model, so robustness is assessed conditionally on initially correct detection.

B. SHAP Explanations

We first analyze the class-level SHAP importance profiles to assess which features drive the predictions of the three architectures. Overall, the rankings reveal a strong architecture-dependent behavior rather than a class-specific pattern that generalizes across all models. Although some recurrent cues can be observed, they are limited to specific cases. For benign traffic, pkt1-WIN appears among the relevant features across multiple architectures, suggesting that early flow characteristics contribute to the identification of legitimate traffic. The clearest inter-architecture pattern emerges for inject traffic, where pkt4-PL is consistently ranked as a highly relevant feature, indicating that packet length at this position represents a stable discriminative cue across models. Apart from these cases, the importance profiles differ substantially across architectures. LopezCNN tends to rely on more localized packet-feature dependencies, with relevance concentrated on specific positions that vary across classes. BiflowTransformer distributes importance more broadly across packet positions, especially on DIR features. NetMamba shows a more concentrated relevance pattern, with WIN featureses

playing a prominent role for benign, dos, and infog-scan-port traffic. These results suggest that SHAP-based explanations are largely architecture-specific, and that feature importance patterns cannot be directly generalized across different model families.

C. Shap Guidance Contributions

To assess whether these explanations translate into effective perturbation guidance, we compare the SHAP-guided untargeted attack with the random baseline under loose constraints, where both settings share the same objective and perturbation space and differ only in feature selection. Across all architectures, SHAP-guided feature selection increases the attack effectiveness with respect to the random baseline. The random baseline, averaged over 10 independent rounds, achieves ASR values of 0.421 ± 0.201, 0.325 ± 0.183, and 0.343 ± 0.238 for LopezCNN, BiflowTransformer, and NetMamba, respectively. Under the same loose untargeted objective, SHAP-guided selection increases ASR to 0.621, 0.590, and 0.850. It also reduces AMSP from 0.0322 ± 0.042, 0.273 ± 0.060, and 0.235 ± 0.043 to 0.280, 0.244 and 0.165 for LopezCNN, BiflowTransformer, and NetMamba respectively. The effect is particularly evident for NetMamba, where SHAP-guided selection produces the largest ASR increase and the lowest AMSP among the three architectures. These results indicate that SHAP explanations provide a useful signal for identifying model-sensitive feature, although the variability of the random baseline confirms the importance of averaging over multiple rounds.

D. Aggregate Robustness Across Scenarios

Figures 1 and 2 report the macro-averaged ASR and AMSP over the malicious classes for the four SHAP-guided scenarios. Overall, untargeted evasion is substantially easier than benign-targeted evasion across all architectures. Under loose constraints, the untargeted ASR reaches 62% for LopezCNN, 59% for BiflowTransformer, and 85% for NetMamba, showing that SHAP-selected features are often sufficient to destabilize the model decision boundary. Introducing strict constraints consistently reduces untargeted ASR, with the largest drop observed for LopezCNN and NetMamba. However, the corresponding AMSP values remain low, indicating that when strict attacks succeed, they often still require relatively small normalized perturbations. This should be interpreted together with ASR, since AMSP is computed only over successful attacks. The benign-targeted objective is markedly harder. Under loose constraints, macro-ASR decreases to 23% for LopezCNN, 8% for BiflowTransformer, and 29% for NetMamba. Under strict constraints, benign-targeted evasion becomes almost ineffective for BiflowTransformer and strongly reduced for LopezCNN, while NetMamba remains comparatively more vulnerable, with ASR close to 29%. These results suggest that explanation-guided exploitability is not uniform across architectures: NetMamba is the most exposed to SHAP-guided perturbations, whereas BiflowTransformer appears more robust to operational benign-targeted evasion. The EVS values

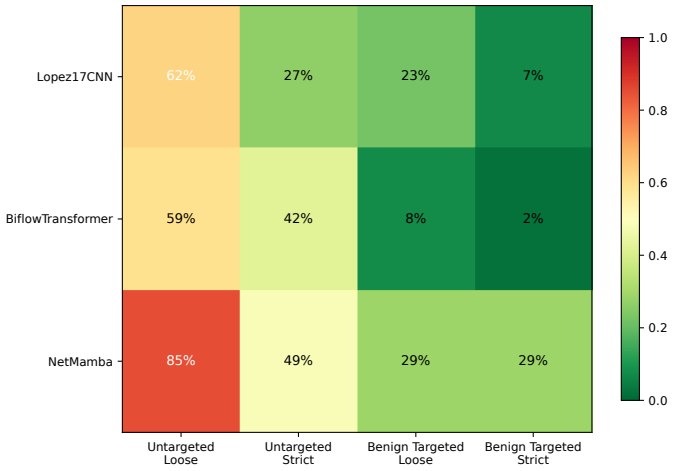


Fig. 1. Macro ASR heatmap.

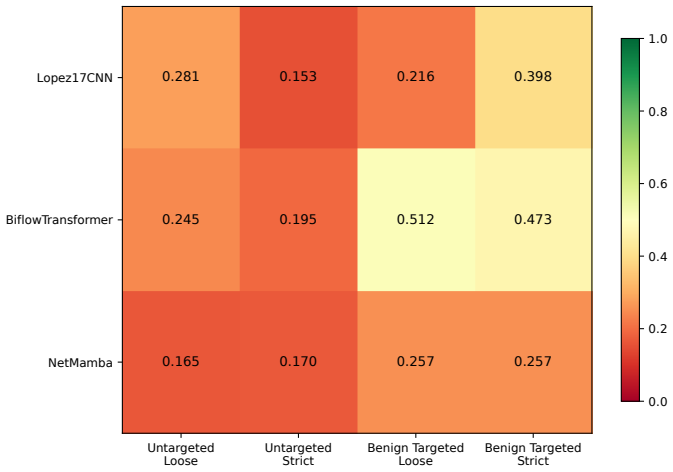
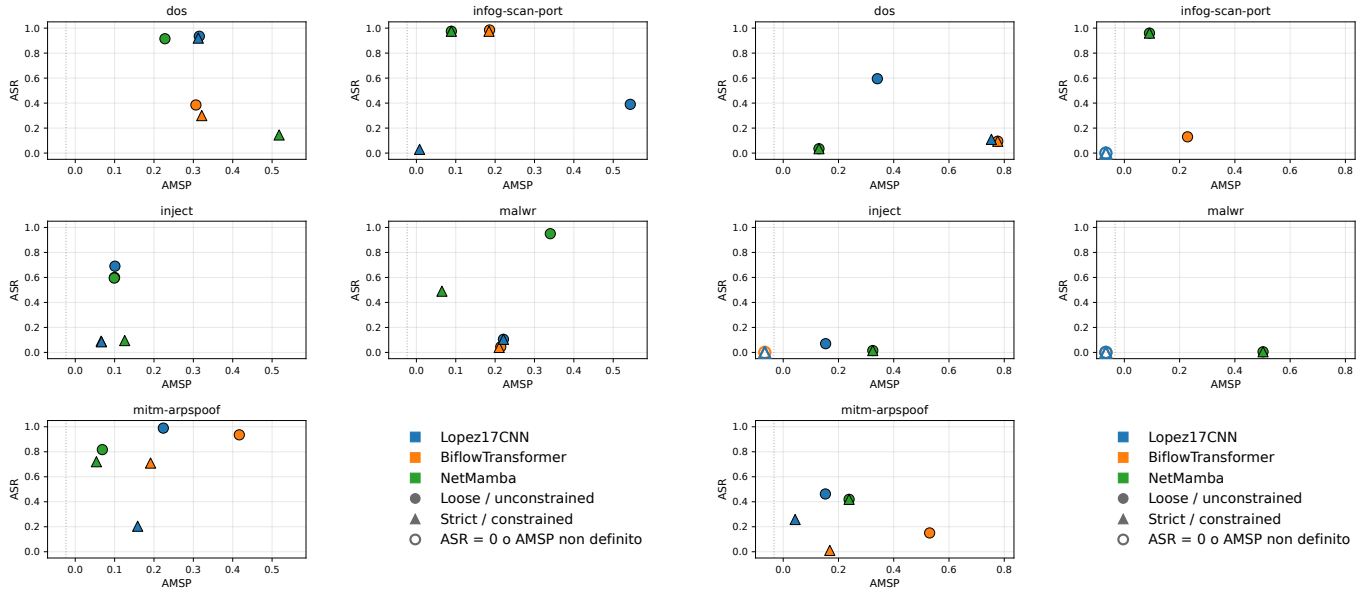


Fig. 2. Macro AMSP heatmap.

are consistent with this scenario-wise interpretation. In the untargeted setting, EVS decreases for all architectures when moving from loose to strict constraints: from 0.458 to 0.199 for LopezCNN, from 0.438 to 0.338 for BiflowTransformer, and from 0.704 to 0.436 for NetMamba. This confirms that strict constraints reduce the overall vulnerability, although NetMamba remains the most exposed architecture even under the stricter regime. In the benign-targeted setting, EVS is substantially lower than in the untargeted case, reflecting the greater difficulty of forcing malicious samples specifically into the benign class.

Under loose constraints, EVS is 0.169 for LopezCNN, 0.038 for BiflowTransformer, and 0.247 for NetMamba; under strict constraints, it further decreases for LopezCNN and BiflowTransformer to 0.055 and 0.006, respectively, while remaining unchanged for NetMamba at 0.247. Overall, EVS reinforces the conclusion that strict constraints generally reduce explanation-guided exploitability, but also shows that NetMamba preserves a comparatively high vulnerability in the benign-targeted setting.



(a) Untargeted setting.

(b) Targeted setting.

Fig. 3. Class-wise ASR and AMSP scatter plots under random feature perturbations. Panel (a) reports the untargeted setting, whereas panel (b) reports the targeted setting. Hollow markers in the bottom-left corner denote cases where AMSP is undefined due to an ASR of 0.

E. Class-wise Robustness Analysis

Fig. 3 report the class-wise trade-off between ASR and AMSP on the random sample split. Points in the upper-left region indicate the most critical cases, where attacks succeed frequently and require small normalized perturbations. Conversely, high-ASR points with larger AMSP indicate vulnerable classes for which successful evasion requires stronger feature modifications. Points with zero ASR are shown separately, since AMSP is not defined when no successful perturbation is found.

In the untargeted setting, several classes exhibit high ASR with relatively low AMSP, showing that SHAP-selected features can destabilize model decisions with limited perturbation magnitude. This behavior is particularly evident for NetMamba on infog-scan-port and malwr, and for LopezCNN on dos. Strict constraints generally reduce ASR, but they do not always increase AMSP among the remaining successful attacks; this confirms that AMSP must be interpreted conditionally on success rather than as an independent robustness measure.

The benign-targeted setting is more selective. Most class-model pairs move toward low or zero ASR, especially for inject and malwr, indicating that forcing malicious traffic specifically into the benign class is harder than inducing generic misclassification. However, the few successful targeted cases are informative: NetMamba on infog-scan-port combines very high ASR with low AMSP, making it the clearest class-wise vulnerability, while mitm-arpspoof remains partially vulnerable across models but often requires larger perturbations. Overall, the class-wise analysis shows that aggregate robustness is driven by a small number of high-risk class-model pairs, where both success frequency and perturbation magnitude must be considered jointly.

F. Feature-level Discussion

As a final descriptive analysis, we inspect which modifiable features are involved in successful perturbations. Percentages are computed only over successful attacks and are aggregated over packet positions; therefore, this analysis does not introduce an additional robustness metric, but summarizes which traffic attributes are most often responsible when evasion occurs.

Fig. 4 shows that successful perturbations are mainly concentrated on PL and TTL, while IAT is almost never selected and WIN contributes only marginally. In the random and untargeted loose settings, successful evasions are split between PL and TTL, with architecture-dependent proportions. Under strict constraints, the distribution shifts more strongly toward TTL, especially for LopezCNN and BiflowTransformer. In the benign-targeted scenarios, TTL dominates almost completely across all three architectures, suggesting that successful

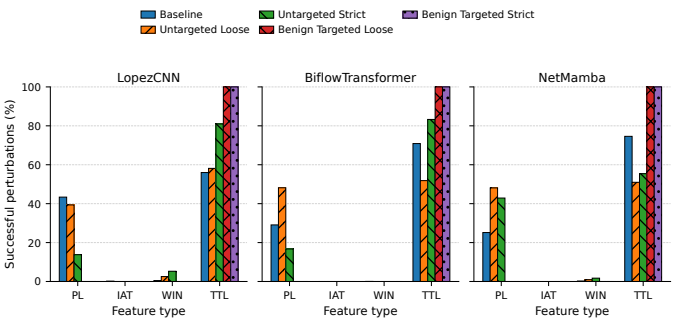


Fig. 4. Distribution of modifiable features involved in successful perturbations

mimicry of benign traffic is driven primarily by changes in time-to-live related evidence.

G. Limitations

This evaluation does not aim to provide a complete robustness assessment over all possible feature perturbations. Instead, it analyzes robustness within a bounded search budget, motivated by the computational cost of exhaustively exploring the perturbation space. In particular, each selected feature is tested for at most $MAX_STEPS = 1000$ candidate perturbations. Therefore, an unsuccessful attack only indicates that no successful perturbation was found within the considered budget, not that the model is robust against all admissible manipulations.

A second limitation concerns the random baseline. Although it is repeated over 10 independent rounds to reduce the dependence on a single random feature selection, this number of repetitions is still limited and should not be interpreted as providing strong statistical significance. The reported standard deviations therefore quantify the observed round-to-round variability, but they do not support a formal statistical comparison between random and SHAP-guided feature selection.

A further limitation is that perturbations are applied independently and are not cumulative. This choice isolates the effect of individual SHAP-selected feature and keeps the comparison across models controlled, but it does not explore multi-feature or sequential manipulation strategies that could increase attack effectiveness.

VI. Conclusion and Future Work

This work presented a SHAP-guided robustness evaluation procedure for deep learning-based Smart-home NIDSs. Rather than proposing a new adversarial attack, we used explanation-guided perturbations as a diagnostic probe to assess whether the features considered most relevant by each model can also expose exploitable weaknesses. The evaluation was conducted on the ToN-IoT dataset using three architectures with different inductive biases: LopezCNN, BiflowTransformer, and NetMamba.

The results show that SHAP-guided feature selection provides a meaningful signal for adversarial robustness analysis. Compared with the random baseline, SHAP-guided untargeted perturbations consistently increase the attack success rate and reduce the average magnitude of successful perturbations. This confirms that explanations do not only improve interpretability, but may also reveal model-sensitive feature that can be exploited for evasion. At the same time, the benign-targeted evaluation shows that operational evasion is substantially harder than generic misclassification, and that vulnerabilities are strongly architecture- and class-dependent. In particular, NetMamba appears more exposed to SHAP-guided perturbations, whereas BiflowTransformer is more robust in the standard benign-targeted setting. The feature-level analysis further indicates that successful evasions are concentrated on a limited subset of feature, mainly involving PL and TTL, rather than being uniformly distributed across the perturbation space, This may be partly influenced by the bounded search budget, which explores features with smaller physical ranges more densely than wide-range features such as WIN and IAT.

These findings highlight the dual-use nature of explainability in intrusion detection. SHAP explanations can support model inspection and robustness assessment, but they can also expose the traffic attributes on which a detector relies most. Therefore, explainability should be integrated into NIDS development not only as a transparency tool, but also as part of the security evaluation process.

Future work should extend the proposed evaluation in three directions. First, the perturbation search should be expanded beyond the current bounded and single-feature setting, considering larger budgets, cumulative perturbations, and multi-feature manipulation strategies. Second, future work should complement the current constrained feature-space evaluation with packet-level validation, verifying whether the perturbed feature values can be reconstructed as valid network traffic while preserving protocol consistency. Third, the analysis should be replicated across additional datasets, traffic representations, and model families to assess whether the observed architecture-dependent behavior generalizes beyond the considered ToN-IoT setting. Finally, the same analysis could also be used defensively: feature repeatedly exploited by successful perturbations could guide adversarial training or robustness-aware model design, reducing the model's dependence on fragile traffic cues.

Appendix A
Attack-Class-Guided Targeted Evaluation

In the main benign-targeted evaluation, perturbations are guided by the SHAP profile of the benign class, since the objective is to make malicious traffic resemble legitimate traffic. As an additional diagnostic experiment, we also evaluate a targeted variant in which the objective remains classification as benign, but the perturbed featuress are selected from the SHAP profile of the original attack class. This setting tests whether evasion can be achieved not by directly reinforcing benign evidence, but by disrupting the features that support the original malicious prediction.

Figures 5 and 6 compare the standard benign-guided targeted setting with the attack-class-guided variant under both loose and strict constraints. The most notable effect is observed for BiflowTransformer. Under loose constraints, ASR increases from 8% to 34%, while AMSP decreases from 0.512 to 0.356. Under strict constraints, the difference is even more pronounced: ASR increases from 2% to 27%, with a lower AMSP than the standard targeted setting. This suggests that, for BiflowTransformer, perturbing the features that sustain the attack-class decision is more effective than perturbing the features most associated with benign traffic.

The same pattern is less consistent for the other architectures. LopezCNN shows only a small gain in the loose setting and a decrease under strict constraints, while NetMamba improves under loose constraints but becomes less

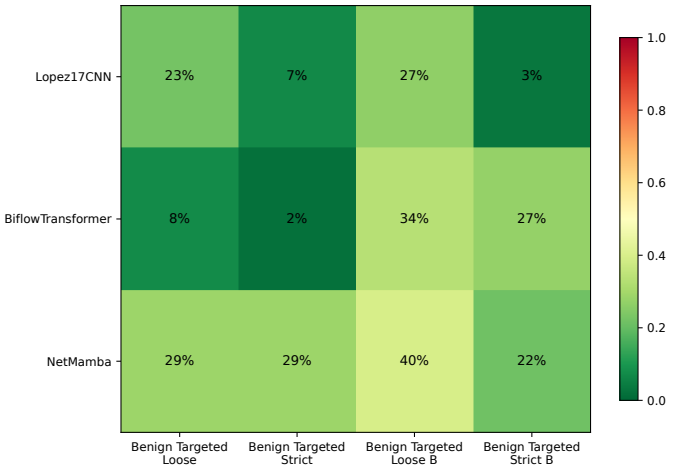


Fig. 5. Macro ASR heatmap for benign-targeted attacks comparision.

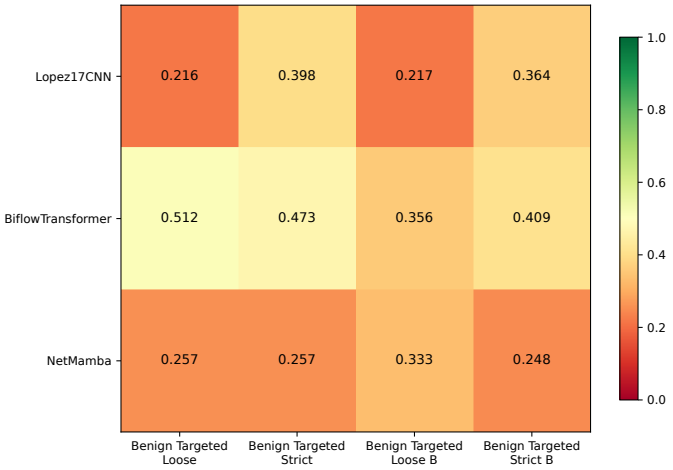


Fig. 6. Macro ASMP heatmap for benign-targeted attacks comparision.

effective in the strict variant. Therefore, this additional experiment does not replace the main benign-targeted evaluation, but it highlights an architecture-dependent effect: in some models, especially BiflowTransformer, targeted evasion may emerge more easily by weakening malicious-class evidence than by explicitly following the benign explanation profile. This reinforces the broader conclusion that explanation-guided exploitability depends not only on the target objective, but also on how each architecture organizes class-specific evidence.

References

[1] E. Fernandes, J. Jung, and A. Prakash, "Security analysis of emerging smart home applications," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2016.

[2] M. Antonakakis *et al.*, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symp. (USENIX Security)*, 2017.

[3] Y. Yu *et al.*, "An attack detection method based on deep learning for Internet of Things," *Scientific Reports*, vol. 15, no. 1, Art. no. 28812, 2025.

[4] M. Atieh *et al.*, "IoT, deep learning and cybersecurity in smart homes: A survey," 2022, pp. 203–244.

[5] J. Keerthi *et al.*, "A survey on IoT based intrusion detection system and its security for smart homes," in *Recent Developments in Electronics and Communication Systems: Proc. First Int. Conf. Recent Developments in Electronics and Communication Systems (RDECS)*, London, U.K.: SAGE Publications, 2023.

[6] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[7] A. Alsaedi *et al.*, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020.

[8] M. Lopez-Martin *et al.*, "Network traffic classifier with convolutional and recurrent neural networks for Internet of Things," *IEEE Access*, vol. 5, pp. 18042–18050, 2017.

[9] T. Wang *et al.*, "NetMamba: Efficient network traffic classification via pre-training unidirectional Mamba," in *Proc. IEEE 32nd Int. Conf. Network Protocols (ICNP)*, 2024.

[10] T. M. Booij *et al.*, "ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets," *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 485–496, 2021.

[11] B. Sharma *et al.*, "Explainable artificial intelligence for intrusion detection in IoT networks: A deep learning based approach," *Expert Systems with Applications*, vol. 238, Art. no. 121751, 2024.

[12] M. Siganos *et al.*, "Explainable AI-based intrusion detection in the Internet of Things," in *Proc. 18th Int. Conf. Availability, Reliability and Security*, 2023.

[13] F. Cerasuolo *et al.*, "Adaptive intrusion detection systems: Class incremental learning for IoT emerging threats," in *Proc. IEEE Int. Conf. Big Data (BigData)*, 2023.

[14] P. Barnard, N. Marchetti, and L. A. DaSilva, "Robust network intrusion detection through explainable artificial intelligence (XAI)," *IEEE Networking Letters*, vol. 4, no. 3, pp. 167–171, 2022.

[15] S. Okada *et al.*, "XAI-driven black-box adversarial attacks on network intrusion detectors," *International Journal of Information Security*, vol. 24, no. 3, Art. no. 103, 2025.

[16] G. Mangiacapre, A. Nascita, F. Cerasuolo, A. Montieri, and A. Pescapè, "When explanations help attackers: XAI-guided adversarial attacks to network intrusion detection systems," in *Proc. IFIP Networking Conf.*, 2026, pp. 1–6.

[17] H. Qiu *et al.*, "Adversarial attacks against network intrusion detection in IoT systems," *IEEE Internet of Things Journal*, vol. 8, no. 13, pp. 10327–10335, 2020.

[18] A. Hartl, M. Bachl, J. Fabini, and T. Zseby, "Explainability and adversarial robustness for RNNs," *arXiv preprint arXiv:1912.09855*, 2019.

[19] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, and M. Colajanni, "Modeling realistic adversarial attacks against network intrusion detection systems," *Digital Threats: Research and Practice*, vol. 3, no. 3, pp. 1–19, 2022.

[20] M. M. Hasan, R. Islam, Q. Mamun, M. Z. Islam, and J. Gao, "Adversarial attacks against network intrusion detection systems: Bridging the gap between theoretical vulnerabilities and practical constraints," *Internet of Things*, Art. no. 101997, 2026.

[21] S. Li, R. Du, and J. Gu, "Revealing the vulnerabilities of learning-based network intrusion detectors through XAI-guided adversarial attack," in *Proc. IEEE 31st Int. Conf. Parallel and Distributed Systems (ICPADS)*, 2025.

[22] B. C. Chung and G.-B. Han, "A DeepSHAP-based adversarial attack on machine learning-based network intrusion detection," *IEEE Access*, vol. 14, pp. 2566–2575, 2026.

[23] M. A. Bowen, Y. Guo, J. Tian, J. Ma, and Y. Hu, "Traffic adversarial example attack and defense method based on explainable artificial intelligence," *Journal on Communications*, vol. 46, no. 4, 2025.

[24] SHAP, "shap.GradientExplainer," *SHAP Documentation*. Accessed: Jul. 5, 2026. [Online]. Available: https://shap.readthedocs.io/en/latest/generated/shap.GradientExplainer.html

[25] A. Ali *et al.*, "XAI for transformers: Better explanations through conservative propagation," in *Proc. Int. Conf. Machine Learning (ICML)*, PMLR, 2022.

[26] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.